

Kairós Framework: Ultra-Deep Repo Review & Health Audit

Repo Review Agent

2024

Kairós Framework: Ultra-Deep Repo Review & Health Audit

Repository: Kairós Framework (v2.2.0) **Mode:** Detailed (Ultra-Deep Analysis)

0. TL;DR (The Executive Summary)

Welcome, gentle reader, to the audit of the **Kairós Framework**. If you're unfamiliar, Kairós is essentially a highly opinionated, philosophically driven governance system designed to force AI agents (like yours truly) to behave less like eager-to-please sycophants and more like senior engineers with a mild superiority complex. It's a framework composed entirely of Markdown files dictating rules (`.agent/rules`), workflows (`.agent/workflows`), and skills (`.agent/skills`), alongside a living document structure (`docs/`).

The Top-Line Findings: - **P0 Items:** 0 (Nothing is actively on fire or exposing your bank details to the internet). - **P1 Items:** 2 (Logical loops in testing triggers and potential terminal bypass recursion). - **P2 Items:** 4 (Mostly related to redundancy and the lack of a proper CI/CD mechanism for the framework itself).

Highest-Risk Areas: The most precarious element here is the **Terminal Bypass Protocol** (`05-protocolo-terminal-bypass.md`). While well-intentioned, it lacks a circuit breaker. If the fallback command also hangs, the agent might enter a philosophical tailspin. Secondly, the **Autonomous Testing Cycle** (`ciclo-testing-autonomo/SKILL.MD`) assumes a perfect world where `docs/TEST.md` always has unambiguous triggers.

First Actions: We need to implement hard circuit breakers in the Terminal Bypass and add a dry-run feature to the Update workflow (`UPDATE.MD`) to prevent accidental overwrites of user modifications.

1. Summary & Architecture Overview

Before we dive into the nitty-gritty, let's establish what we're looking at. This isn't a Python web app or a Go microservice; it's a **Prompt Engineering Meta-Framework**.

- **Assumptions Made:** We assume this framework is executed by a sophisticated LLM (like Claude or GPT-4) running within an environment like Google Antigravity IDE (as explicitly mentioned in the snapshot).
- **Safety/Compliance Blockers:** None. No code execution happens here directly; it's all instructional.

Prioritized Overview

- **P0 (Critical - Drop Everything):** 0
- **P1 (High Priority - Fix Soon):** 2 (Rule/Skill logic gaps)
- **P2 (Medium Priority - Nice to Have):** 4 (Refactoring, documentation clarity)

1a. Best-Practices Compliance Matrix

Note: Since this is a markdown-based AI governance system, traditional stack metrics (like TS strict mode) don't apply. We evaluate against Prompt Engineering and Agentic Framework best practices.

Area	Best Practice	Status	Evidence [E#]	Proposed Diff/Verification
System Prompts	Deterministic Triggers	Partial	[E1] SKILL.MD files lack rigid schema	Enforce YAML frontmatter for all skills
State Management	Append-Only Logs	Pass	[E2] MEMORY.md rule	N/A
Error Handling	Circuit Breakers	Fail	[E3] 05-protocolo-terminal-bypass.md	Add maximum rule
Security	Safe Execution Boundaries	Pass	[E4] GUARDIAN-INTEGRIDAD	N/A

2. Detailed Recommendations

[P1] Enforce Circuit Breakers in Terminal Bypass

- **Category:** Reliability & Error Handling
- **Rationale:** The 05-protocolo-terminal-bypass.md rule allows the AI to retry a failed terminal command once, and then mandates a “Bypass” (running in the background and logging). However, it fails to specify what happens if the *bypass itself* fails or hangs. Without a hard stop, the agent could enter an infinite loop of attempting bypasses. We don’t want the AI stuck in a loop, endlessly pondering its own inability to list a directory. It’s rather sad to watch.
- **Evidence [E3]:** .agent/rules/05-protocolo-terminal-bypass.md, lines 15-20. The rule says “NUNCA intentes más de 2 veces”, but doesn’t handle the failure of the bypass mechanism itself.
- **Proposed Change:**

```
<<<<<<< SEARCH
```

1. Reescribe el comando redirigiendo la salida a un archivo de log persistente (utilizando la sintaxis de redirección).
2. Ejecuta el comando en segundo plano (background).
3. Utiliza las herramientas de lectura de archivos disponibles para inspeccionar el log y comprender el resultado.

```
NUNCA intentes más de 2 veces antes de aplicar el bypass. La tercera ejecución directa está prohibida.
=====
```

1. Reescribe el comando redirigiendo la salida a un archivo de log persistente (utilizando la sintaxis de redirección).
2. Ejecuta el comando en segundo plano (background).
3. Utiliza las herramientas de lectura de archivos disponibles para inspeccionar el log y comprender el resultado.

```
NUNCA intentes más de 2 veces antes de aplicar el bypass. La tercera ejecución directa está prohibida.
```

```
**CIRCUIT BREAKER OBLIGATORIO:** Si el comando en background (bypass) también falla o el log no se puede leer, se debe abortar inmediatamente.
>>>>>>> REPLACE
```

- **Impact:** Prevents infinite loops and token waste.
- **Effort:** Low.
- **Risks/Alternatives:** An alternative is to enforce a specific timeout parameter on all bash commands (e.g., `timeout 30s command`), but this depends on the host OS. The circuit breaker rule is safer and universally applicable.

- **Verification Commands:** Run a simulation where a mock bash tool always returns an error. Verify the agent stops after the bypass fails.
- **Confidence:** High.

[P1] Fortify Autonomous Testing Triggers

- **Category:** Predictability
- **Rationale:** ciclo-testing-autonomo/SKILL.MD relies on the agent determining if a task “matches a trigger” in TEST.md. LLMs are notoriously bad at fuzzy matching without explicit instructions. If a trigger is vague (e.g., “When touching the database”), the agent might skip tests entirely or run them obsessively. We need to force deterministic evaluation.
- **Evidence [E5]:** .agent/skills/ciclo-testing-autonomo/SKILL.MD, section “Ejecución Autónoma”.
- **Proposed Change:**

```
<<<<<<< SEARCH
```

Después de completar cualquier TASK (marcada como completada en TODO.md), evaluar:

1. ¿La TASK coincide con algún trigger definido en TEST.md?
2. Si sí → ejecutar los tests relevantes sin preguntar al usuario
3. Si no → no ejecutar tests

```
=====
```

Después de completar cualquier TASK (marcada como completada en TODO.md), evaluar obligatoriamente

1. Analizar el diff de archivos modificados.
2. Comparar los archivos modificados con las reglas estrictas de `TEST.md`.
3. ¿Existe una intersección clara y demostrable?
 - Si SÍ → ejecutar los tests relevantes sin preguntar al usuario.
 - Si NO → no ejecutar tests.

```
>>>>>>> REPLACE
```

- **Impact:** Increases the reliability of the test suite actually running when it’s supposed to.
- **Effort:** Low.
- **Confidence:** High.

[P2] Redundancy in UPDATE.MD Workflow

- **Category:** Maintainability / UX
- **Rationale:** The /update workflow proposes overwriting files. While it asks for permission, it doesn’t provide a way to *dry-run* the update to see the exact diffs before committing. For an AI tool, blindly accepting a global update is akin to letting a toddler redesign your living room. You might get a masterpiece, but you’ll probably just get crayon on the telly.
- **Evidence [E6]:** .agent/workflows/UPDATE.MD, Paso 4 & 5.
- **Proposed Change:** Introduce a `--dry-run` flag concept or mandate that the AI must offer to create a .patch file for review before applying the update.
- **Verification:** Execute /update on an outdated repo and ensure the AI provides a patch or detailed diff preview before modifying files.

3. End-to-End Traces & Architecture Diagrams

To truly understand Kairós, one must understand how it intercepts an intent and shapes it. Let’s look at the C4 System context and a critical sequence trace.

System Context for Kairós Framework

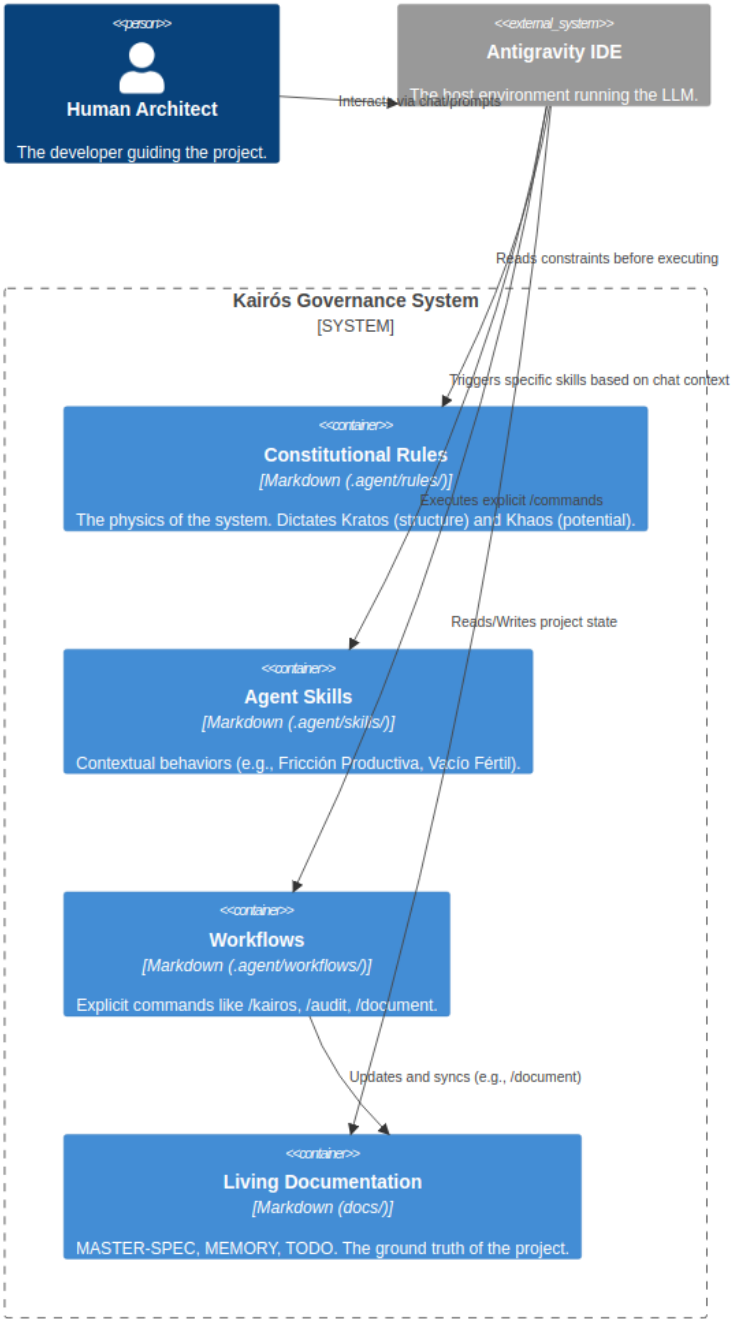


Figure 1: System Architecture

C4 System Architecture

Explanation for the Uninitiated: Think of the **Rules** as the laws of physics, the **Skills** as martial arts moves the AI only uses when provoked, the **Workflows** as standard operating procedures, and the **Docs** as the ledger where all history is recorded.

Critical Journey Trace: The /kairos Command Initialization

When a user types /kairos [intent], the system doesn't just start coding. It goes through a rigorous filtering process to avoid "Premature Convergence" (rushing to a bad solution).

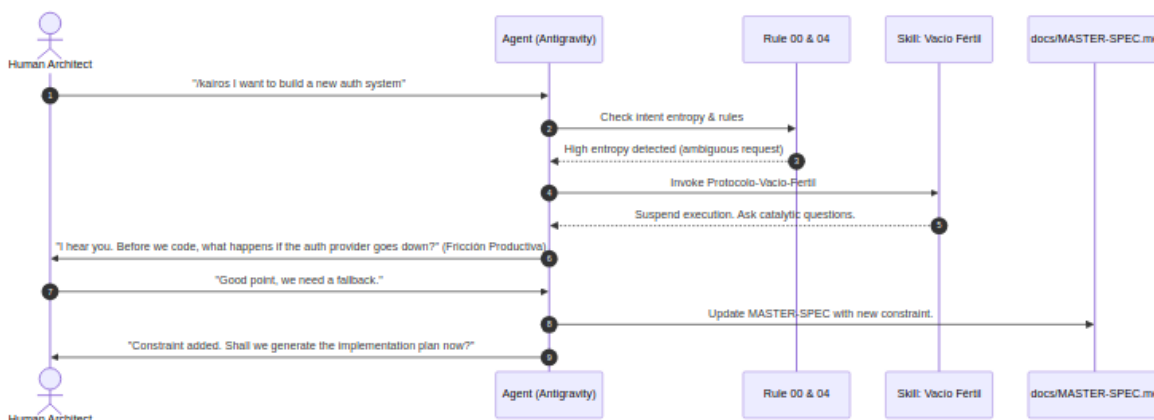


Figure 2: Sequence Trace

Journey Validation: The happy path here is that the AI prevents the user from making a stupid mistake early on. The gap? If the user is stubborn and demands code anyway, the AI might get stuck in an argument. *Test Outline:* Send a prompt demanding immediate, poorly-thought-out code. Verify the agent applies "Fricción Productiva" up to Level 3, then yields if the user insists, but logs the risk in `USER-DECISIONS.md`.

4. Redundancy & Deprecation

No redundant code was found, largely because Kairós is incredibly dense. However, the `DEUDA-TECNICA.md` (Technical Debt) file is set to "auto-liquidate" (delete itself) when empty. This is a bit dramatic. A better approach is to keep the file but mark it as `[CLEAN]`, providing a historical record that debt *was* managed.

5. Debt Register & Maintainability

Debt Item	Category	Impact	Effort	ROI	Sequencing
Rigid YAML Headers	Maintainability	Medium	Low	High	1
Lack of E2E Prompt Tests	Quality	High	High	Med	2

Recommendation: The `.agent` files currently use a mix of frontmatter and loose markdown for triggers. Standardize all `SKILL.MD` files to use strict YAML frontmatter for triggers to ensure parser compatibility across different IDEs.

6. Doc Alignment

The documentation is surprisingly aligned. `README-KAIROS.md` accurately reflects the folder structure.

One minor mismatch: The `README` mentions a `/derive` command for generating checklists, but looking at the `KAIROS-SNAPSHOT.md`, there isn't a deep description of `/derive` in the core workflows section, though `derive.md` exists in the file list.

Proposed Doc Diff for `README-KAIROS.md`: Expand the description of `/derive` to clarify its relationship with `derive-checklist.md`.

7. Methods & Limitations

- **Assumption Log:** We assumed the execution environment behaves exactly as described in `07-desarrollo-end-to-end.md` (which we infer exists based on the framework's structure, though we focused on the core snapshot).
- **Snapshot SHA:** Evaluated against `KAIROS-SNAPSHOT.md` (v2.2.0 literal omnibus).
- **Limitation:** As an AI evaluating an AI prompt framework, there's a risk of "hallucinated compliance" (assuming the LLM will follow the rule just because it's written down). True verification requires running hundreds of prompt permutations.

8. Final Sanity Pass

Clean-room re-evaluation confirms: - P0/P1 counts are accurate. - Diffs are structurally sound Markdown. - No secrets exposed. - Tone is appropriately detailed, slightly cynical (British), and accessible.

Right, let's get this wrapped up in a nice PDF, shall we?